

Code Explanation

Batch Processing Test Suite Explanation

The provided code is a test suite for a batch processing system written in Python using the PySpark library. It includes tests for data cleaning, order summary creation, and customer aggregate spend calculation.

Overview of the Code

The test suite is designed to test the functionality of three main functions: `clean\_data`, `create\_or\_update\_ordersummary`, and `create\_customeraggregatespend`. These functions are part of a batch processing system that handles customer and order data. The test suite creates a Spark session, generates sample data, and then tests the functions using this data.

Step 1: Setting Up the Test Environment

This step involves creating a Spark session and setting up the necessary environment for testing. It includes creating a temporary directory for test data and initializing the test catalog and schema.

```
@classmethod
def setUpClass(cls):
    cls.spark = (SparkSession.builder
                  .appName("TestCustomerOrderProcessing")
                  .master("local[*]")
                  .config("spark.sql.extensions", "io.delta.sql.DeltaS
parkSessionExtension")
                  .config("spark.sql.catalog.spark_catalog", "org.apac
he.spark.sql.delta.catalog.DeltaCatalog")
                  .getOrCreate())

    cls.temp_dir = tempfile.mkdtemp()
    cls.catalog = "test_catalog"
    cls.schema = "test_schema"

    cls.spark.sql(f"CREATE DATABASE IF NOT EXISTS {cls.catalog}.{cls
.schema}")

    cls.create_sample_data()
```

Step 2: Creating Sample Data

This step generates sample customer and order data to be used for testing. The data includes valid records as well as records with null values and duplicates to test the data cleaning functionality.

```
@classmethod
def create_sample_data(cls):
    customer_data = [
        (1, "John Doe", "john@example.com", "North"),
        (2, "Jane Smith", "jane@example.com", "South"),
        (3, "Bob Johnson", "bob@example.com", "East"),
        (4, "Alice Brown", "alice@example.com", "West"),
        (5, "Charlie Davis", None, "North"),
        (5, "Charlie Davis", "charlie@example.com", "North"),
        (None, "Invalid", "invalid@example.com", "Unknown")
    ]
```

```

]

customer_schema = StructType([
    StructField("CustId", IntegerType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_df = cls.spark.createDataFrame(customer_data, schema=customer_schema)

order_data = [
    (101, "Laptop", 1200.0, 1, "2023-01-15", 1),
    (102, "Phone", 800.0, 2, "2023-01-20", 2),
    (103, "Tablet", 500.0, 1, "2023-02-05", 3),
    (104, "Monitor", 300.0, 2, "2023-02-10", 4),
    (105, "Keyboard", 50.0, 3, "2023-03-01", 1),
    (106, "Mouse", 25.0, 2, "2023-03-05", 2),
    (107, "Headphones", None, 1, "2023-03-10", 3),
    (107, "Headphones", 100.0, 1, "2023-03-10", 3),
    (None, "Invalid", 10.0, 1, "2023-04-01", 5)
]

order_schema = StructType([
    StructField("OrderId", IntegerType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", StringType(), True),
    StructField("CustId", IntegerType(), True)
])

cls.order_df = cls.spark.createDataFrame(order_data, schema=order_schema)

```

Step 3: Testing Data Cleaning Functionality

This step tests the `clean\_data` function, which is responsible for cleaning the customer and order data. The test checks if the function correctly removes null values and duplicates.

```

def test_clean_data(self):
    cleaned_customer_df, cleaned_order_df = clean_data(self.customer_df, self.order_df)

    self.assertEqual(cleaned_customer_df.count(), 4)
    self.assertEqual(cleaned_customer_df.filter(F.col("CustId") == 5).count(), 1)

    self.assertEqual(cleaned_order_df.count(), 6)
    self.assertEqual(cleaned_order_df.filter(F.col("OrderId") == 107).count(), 1)

    order_with_laptop = cleaned_order_df.filter(F.col("ItemName") == "Laptop").collect()[0]
    self.assertEqual(order_with_laptop["TotalAmount"], 1200.0 * 1)

```

Step 4: Testing Order Summary Creation

This step tests the `create\_or\_update\_ordersummary` function, which creates or updates the order summary table. The test checks if the table is created correctly and if the SCD Type 2 columns are present.

```
def test_create_or_update_ordersummary(self):
    cleaned_customer_df, cleaned_order_df = clean_data(self.customer_df, self.order_df)

    create_or_update_ordersummary(self.spark, cleaned_customer_df, cleaned_order_df, self.catalog, self.schema)

    ordersummary_df = self.spark.table(f"{self.catalog}.{self.schema}.ordersummary")
    self.assertTrue(ordersummary_df.count() > 0)

    self.assertTrue("IsActive" in ordersummary_df.columns)
    self.assertTrue("StartDate" in ordersummary_df.columns)
    self.assertTrue("EndDate" in ordersummary_df.columns)
```

Step 5: Testing Customer Aggregate Spend Calculation

This step tests the `create\_customeraggregatespend` function, which calculates the customer aggregate spend. The test checks if the table is created correctly and if the aggregation is done correctly.

```
def test_create_customeraggregatespend(self):
    cleaned_customer_df, cleaned_order_df = clean_data(self.customer_df, self.order_df)
    create_or_update_ordersummary(self.spark, cleaned_customer_df, cleaned_order_df, self.catalog, self.schema)

    create_customeraggregatespend(self.spark, self.catalog, self.schema)

    agg_df = self.spark.table(f"{self.catalog}.{self.schema}.customeraggregatespend")
    self.assertTrue(agg_df.count() > 0)

    self.assertTrue("Name" in agg_df.columns)
    self.assertTrue("TotalAmount" in agg_df.columns)
    self.assertTrue("Date" in agg_df.columns)

    john_agg = agg_df.filter(F.col("Name") == "John Doe").collect()
    self.assertEqual(len(john_agg), 2)
```